МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Факультет "Информационные технологии"

Кафедра "Информатика"

Лабораторная работа 2 · Вариант 12

«РАЗРАБОТКА И ИССЛЕДОВАНИЕ АЛГОРИТМОВ СОРТИРОВКИ И СЛОЖНОЙ РЕКУРСИИ»

по дисциплине

«Информационные технологии и программирование»

Выполнил: студент группы БСТ2501

Максимов М. М.

Проверил: доцент Карауш А. С.

Москва 2026 г.

1. Постановка задачи

Цель работы: изучить алгоритмы сортировки массивов и разобраться, как работает рекурсия на примере метода Хоара.

Вариант 12: массив чисел типа float, сортировка вставками. Кроме неё нужно реализовать быструю сортировку без рекурсии и рекурсивную быструю сортировку методом Хоара.

Для проверки нужно сгенерировать 50 случайных чисел в заданном диапазоне и отсортировать одинаковый исходный массив тремя способами. Затем сравнить время работы методов при N = 1000, 10000, 100000, 500000 и 1000000. Время вывести в миллисекундах.

Диапазон для варианта 12

В записи [1x0x0; 3x70x] вместо первого x подставляется 1, вместо второго — 2. Получается диапазон [11020; 31702]. Числа создаются по формуле:

```
x = 11020 + (31702 - 11020) *
    ((float)rand() / RAND_MAX);
```

Я использовал srand(12), чтобы при повторном запуске на этой системе получались те же исходные данные. Значения хранятся как float, а при выводе округляются до двух знаков после запятой.

Программа написана на языке C. Исходный файл — sorting.c. Для каждого размера массива все три метода получают отдельную копию одного и того же массива.

2. Алгоритмы решения задачи

2.1. Сортировка вставками

1. Считать первый элемент отсортированной частью массива. Начиная со второго элемента, запоминать текущее значение x.

2. Сравнивать x с элементами слева. Пока элемент больше x, сдвигать его на одну позицию вправо.

3. Записать x на освободившееся место. Повторять действия до конца массива.

На уже отсортированном массиве метод делает около N сравнений. Для случайного и обратного порядка время растёт как $O(N^2)$. Дополнительный массив не нужен: память O(1).

```
Вставками:
 11680.87  12146.66  12784.70  12833.61  13308.77
 13400.89  13538.98  13707.12  13713.33  13898.82
 14858.47  15094.49  15356.16  16935.20  17455.52
 17776.98  18569.71  18837.68  19541.68  19586.10
 20123.76  20188.29  20858.48  20954.68  21175.58
 21355.29  21702.63  23466.09  23551.81  24245.53
 24279.18  24298.01  25117.22  25257.37  25424.67
 26103.48  27083.85  27093.65  27267.79  27760.65
 27776.35  27928.65  27963.52  28052.54  29522.52
 29529.63  30141.54  30599.73  30714.70  31429.81
```

Рисунок 1 — Результат сортировки вставками (N = 50)

2.2. Быстрая сортировка методом Хоара

1. Выбрать опорное значение из середины текущего отрезка. Начать просмотр с двух концов.

2. Слева искать элемент, который не меньше опорного, справа — элемент, который не больше опорного. Если индексы ещё не встретились, обменять элементы.

3. После встречи индексов вернуть границу p. Рекурсивно отсортировать отрезки [left; p] и [p + 1; right]. Завершать рекурсию, если в отрезке меньше двух элементов.

Обе части становятся короче исходного отрезка. Элементы, равные опорному, тоже перемещаются, поэтому повторы не мешают завершению алгоритма. В среднем время O(N log N), в худшем случае $O(N^2)$. Память стека в среднем O(log N), в худшем случае O(N).

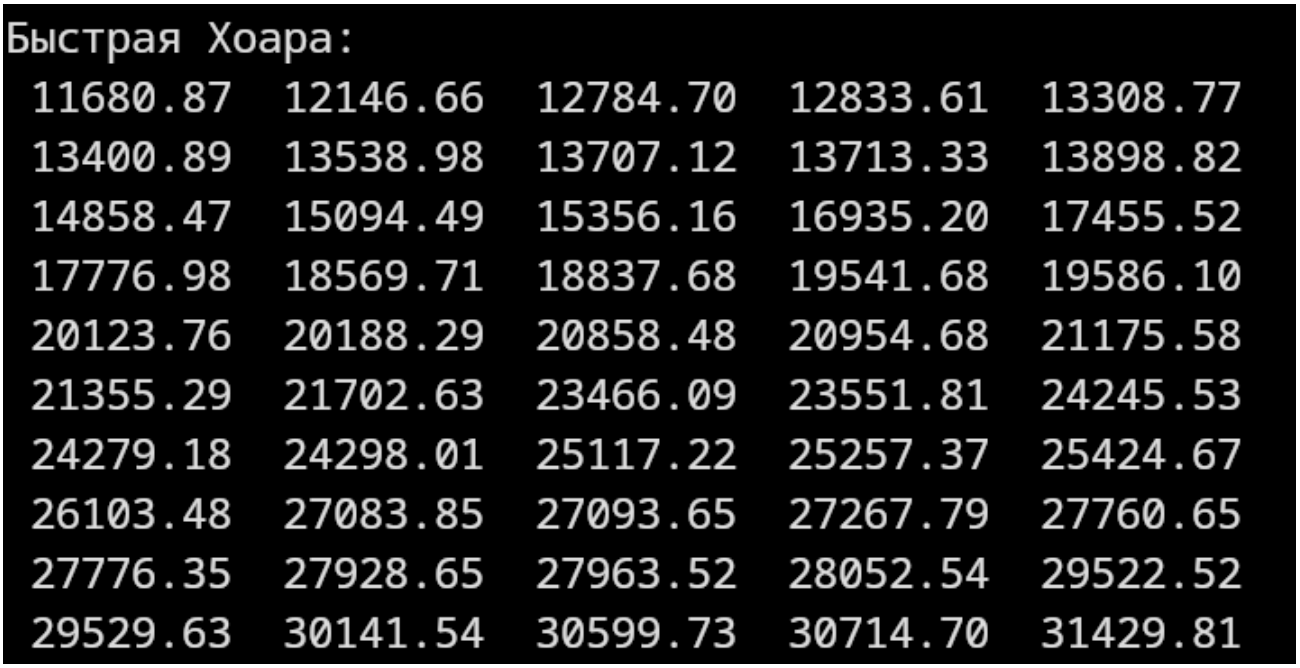


Рисунок 2 — Результат быстрой сортировки методом Хоара (N = 50)

2.3. Быстрая сортировка без рекурсии

Используется то же разбиение Хоара. Вместо рекурсивных вызовов границы отрезков записываются в собственный стек. Больший отрезок откладывается, а меньший обрабатывается сразу. Когда текущий отрезок отсортирован, из стека берётся следующий. При пустом стеке работа заканчивается.

Для размеров типа int достаточно стека из 64 пар границ. Благодаря обработке меньшего отрезка первым дополнительная память O(log N). Средняя и худшая оценки времени совпадают с рекурсивной версией.

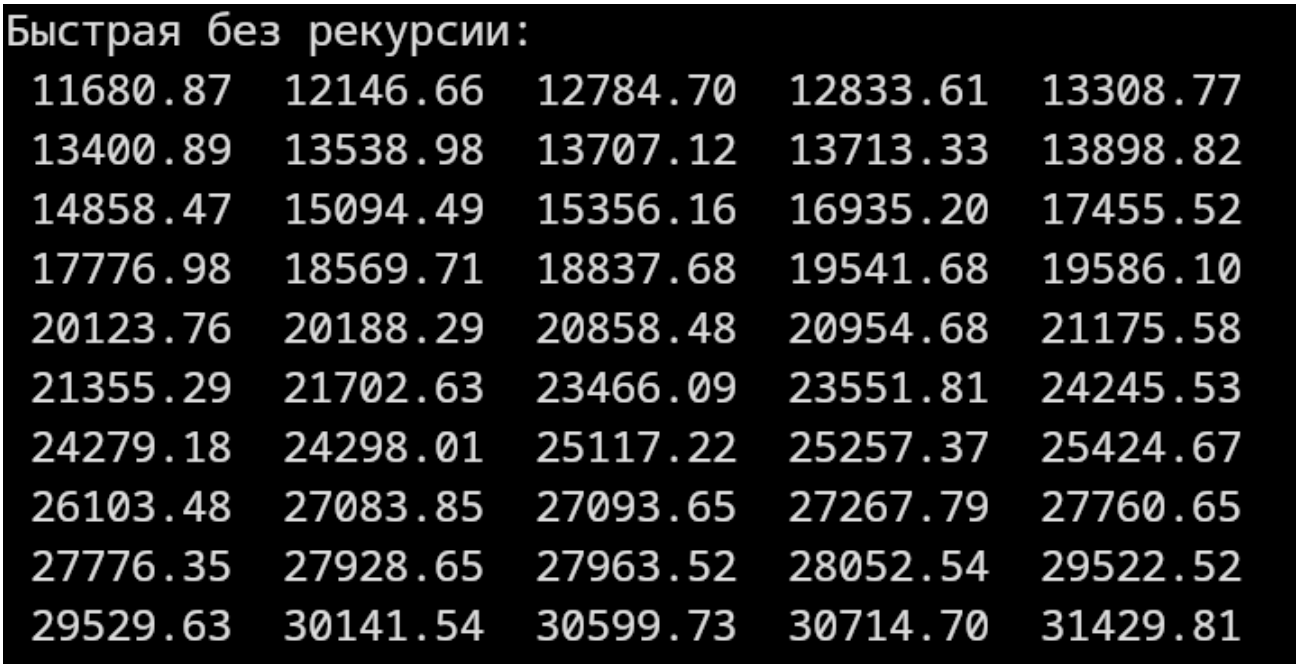


Рисунок 3 — Результат быстрой сортировки без рекурсии (N = 50)

3. Контрольный тест

3.1. Массив из 50 элементов

Я запустил программу с seed = 12. Ниже приведён исходный массив и результат сортировки вставками. Обе быстрые сортировки выдали тот же результат.

Исходный массив:

```
27267.79  20123.76  13400.89  28052.54  25257.37
13707.12  17455.52  21702.63  24298.01  21175.58
12784.70  19586.10  16935.20  12833.61  18569.71
20954.68  24245.53  13713.33  27093.65  29529.63
27963.52  23466.09  18837.68  29522.52  15094.49
14858.47  30599.73  27083.85  11680.87  25117.22
27760.65  27928.65  13538.98  30141.54  24279.18
27776.35  12146.66  30714.70  17776.98  25424.67
20188.29  19541.68  13308.77  26103.48  21355.29
20858.48  15356.16  13898.82  23551.81  31429.81
```

После сортировки:

```
11680.87  12146.66  12784.70  12833.61  13308.77
13400.89  13538.98  13707.12  13713.33  13898.82
14858.47  15094.49  15356.16  16935.20  17455.52
17776.98  18569.71  18837.68  19541.68  19586.10
20123.76  20188.29  20858.48  20954.68  21175.58
21355.29  21702.63  23466.09  23551.81  24245.53
24279.18  24298.01  25117.22  25257.37  25424.67
26103.48  27083.85  27093.65  27267.79  27760.65
27776.35  27928.65  27963.52  28052.54  29522.52
29529.63  30141.54  30599.73  30714.70  31429.81
```

Все 50 чисел расположены по возрастанию. Программа сравнила каждый элемент с результатом библиотечной qsort: для всех трёх методов получено OK. Сравнение проводится по исходным float, без округления для вывода.

3.2. Сборка и запуск

```
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic \
    sorting.c -o sorting
./sorting > results.txt
```

Первая команда собирает программу с оптимизацией. Запуск ./sorting выводит контрольный массив из 50 элементов и результаты его сортировки, затем измеряет время на всех пяти размерах и создаёт results.csv. Компилятор не выдал предупреждений.

4. Результаты работы программы

4.1. Условия измерения

Измерения выполнены на компьютере с процессором AMD Ryzen 5 3600, в Linux x86-64. Компилятор GCC 15.3.1, ключ оптимизации -O2. Сортировки выполнялись последовательно в одном потоке.

Для каждого N создаётся случайный массив с seed = 12. Перед запуском очередного метода он копируется в рабочий массив. Время измеряется функцией clock_gettime(CLOCK_MONOTONIC) непосредственно до и после вызова сортировки. Генерация, копирование, вывод и сравнение с qsort в замер не входят. Работа собственного стека входит во время нерекурсивной сортировки.

После перезапуска среды замер для N = 1000000 выполнен отдельно с теми же генератором, seed, алгоритмами и параметрами компиляции. Предыдущие четыре замера сохранены.

В таблице приведён один фактический замер для каждой пары «метод — размер». После каждого замера результат полностью сверялся с qsort. На всех размерах получено OK. Из-за нагрузки компьютера при повторном запуске время может отличаться.

4.2. Сравнение времени сортировок

Таблица 1 — Время выполнения алгоритмов, мс

Вид сортировки	1000	10000	100000	500000	1000000
Быстрая без рекурсии	0,057	0,758	8,168	45,882	95,867
Быстрая, метод Хоара	0,059	0,731	8,788	46,058	96,095
Вставками (вариант 12)	0,140	13,160	1329,678	32927,683	137315,118

Фрагмент вывода программы:

```
1000; 0.057; 0.059; 0.140; OK
10000; 0.758; 0.731; 13.160; OK
100000; 8.168; 8.788; 1329.678; OK
500000; 45.882; 46.058; 32927.683; OK
1000000; 95.867; 96.095; 137315.118; OK
```

При N = 1000000 сортировка вставками заняла 137,32 с, В этом запуске она оказалась примерно в 1432 раз медленнее более быстрой из двух реализаций быстрой сортировки,

5. Выводы по работе

В работе я реализовал сортировку вставками для массива float и две версии быстрой сортировки. На контрольном массиве из 50 чисел все три метода дали одинаковый результат. Сравнение с qsort подтвердило правильность сортировки на использованных данных.

Сортировка вставками проста, но на больших случайных массивах работает медленно. При увеличении N в 10 раз число сдвигов в среднем увеличивается примерно в 100 раз. Полученные времена показывают этот рост. Быстрые сортировки на исследованных массивах требуют значительно меньше времени.

У двух быстрых сортировок используется одно разбиение, поэтому их времена близки. В рекурсивной версии границы хранятся в стеке вызовов, а в версии без рекурсии — в отдельном массиве. Один запуск не позволяет сделать вывод, что одна версия всегда быстрее другой.

6. Листинг программы

Файл sorting.c. В листинге приведена программа для контрольного теста на 50 элементах и измерения времени сортировок.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define LOW 11020.0f
#define HIGH 31702.0f
#define SEED 12

typedef void (*Sort)(float *, int);

void insertion(float *a, int n)
{
    for (int i = 1; i < n; i++) {
        float x = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > x) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = x;
    }
}

/* Разбиение Хоара: опорным выбирается средний элемент. */
int partition(float *a, int left, int right)
{
    float pivot = a[left + (right - left) / 2];
    int i = left - 1, j = right + 1;
    for (;;) {
        do { i++; } while (a[i] < pivot);
        do { j--; } while (a[j] > pivot);
        if (i >= j) return j;
        float t = a[i]; a[i] = a[j]; a[j] = t;
    }
}

void hoare(float *a, int left, int right)
{
    if (left >= right) return;
    int p = partition(a, left, right);
    hoare(a, left, p);
    hoare(a, p + 1, right);
}

void recursive_sort(float *a, int n)
{
    hoare(a, 0, n - 1);
}
```

```
void iterative_sort(float *a, int n)
{
    /* Откладываем больший отрезок, обрабатываем меньший.
       Поэтому для int-размеров достаточно 64 пар границ. */
    int ls[64], rs[64], top = 0;
    int left = 0, right = n - 1;
    for (;;) {
        while (left < right) {
            int p = partition(a, left, right);
            if (p - left < right - p - 1) {
                ls[top] = p + 1; rs[top++] = right;
                right = p;
            } else {
                ls[top] = left; rs[top++] = p;
                left = p + 1;
            }
        }
        if (top == 0) break;
        left = ls[--top]; right = rs[top];
    }
}

void fill(float *a, int n)
{
    for (int i = 0; i < n; i++)
        a[i] = LOW + (HIGH - LOW) * ((float)rand() / RAND_MAX);
}

void print_array(const float *a, int n)
{
    for (int i = 0; i < n; i++)
        printf("%9.2f%s", a[i], (i + 1) % 5 == 0 ? "\n" : " ");
    if (n % 5) putchar('\n');
}

int compare(const void *a, const void *b)
{
    float x = *(const float *)a, y = *(const float *)b;
    return (x > y) - (x < y);
}

int equal(const float *a, const float *b, int n)
{
    for (int i = 0; i < n; i++)
        if (a[i] != b[i]) return 0;
    return 1;
}

double now_ms(void)
{
    struct timespec t;
    if (clock_gettime(CLOCK_MONOTONIC, &t) != 0) {
        perror("clock_gettime"); exit(EXIT_FAILURE);
    }
    return t.tv_sec * 1000.0 + t.tv_nsec / 1000000.0;
```

```
}

int main(void)
{
    Sort sorts[] = {iterative_sort, recursive_sort, insertion};
    const char *names[] = {"Быстрая без рекурсии", "Быстрая Хоара",
                           "Вставками"};
    int sizes[] = {1000, 10000, 100000, 500000, 1000000};
    setvbuf(stdout, NULL, _IOLBF, 0);
    srand(SEED);
    printf("Вариант 12. float. Диапазон [%.0f; %.0f]. seed=%d\n",
           LOW, HIGH, SEED);
    float source[50], work[50], reference[50];
    fill(source, 50);
    memcpy(reference, source, sizeof source);
    qsort(reference, 50, sizeof(float), compare);
    puts("\nИсходный массив, N=50:"); print_array(source, 50);
    for (int k = 0; k < 3; k++) {
        memcpy(work, source, sizeof source);
        sorts[k](work, 50);
        printf("\n%s:\n", names[k]); print_array(work, 50);
        if (!equal(work, reference, 50)) return EXIT_FAILURE;
        puts("Совпадение с qsort: OK");
    }
    FILE *csv = fopen("results.csv", "w");
    if (!csv) { perror("results.csv"); return EXIT_FAILURE; }
    fprintf(csv, "N,iterative_ms,hoare_ms,insertion_ms\n");
    puts("\nN; без рекурсии, мс; Хоара, мс; вставками, мс");
    for (int s = 0; s < 5; s++) {
        int n = sizes[s];
        float *a = malloc(n * sizeof(float));
        float *b = malloc(n * sizeof(float));
        float *ref = malloc(n * sizeof(float));
        if (!a || !b || !ref) {
            free(a); free(b); free(ref); fclose(csv);
            fputs("Недостаточно памяти\n", stderr); return EXIT_FAILURE;
        }
        srand(SEED); fill(a, n);
        memcpy(ref, a, n * sizeof(float));
        qsort(ref, n, sizeof(float), compare);
        double times[3];
        for (int k = 0; k < 3; k++) {
            memcpy(b, a, n * sizeof(float));
            double start = now_ms();
            sorts[k](b, n);
            times[k] = now_ms() - start;
            if (!equal(b, ref, n)) {
                fputs("Ошибка сортировки\n", stderr);
                free(a); free(b); free(ref); fclose(csv);
                return EXIT_FAILURE;
            }
        }
        printf("%d; %.3f; %.3f; %.3f; OK\n",
               n, times[0], times[1], times[2]);
        fprintf(csv, "%d,%.3f,%.3f,%.3f\n",
                n, times[0], times[1], times[2]);
        fflush(csv);
        free(a); free(b); free(ref);
```

```
    }
    fclose(csv);
    return EXIT_SUCCESS;
}
```